

Healthcare Appointment & Prescription Management System (HAPMS)

Milestones 2–5 — Version Control Document

Yahya Raees Khan | Eshal Khan
BSAI (B) | Database Systems Lab
Instructor: Ali Hassan

Milestone 2 — ERD Design & Normalization

Overview

The HAPMS schema was reviewed against 1NF, 2NF, and 3NF. Changes made and justifications are documented per table below.

Table	1NF	2NF	3NF	Changes Made
USER	OK	OK	OK	All columns atomic; single PK; no transitive deps.
DOCTOR	OK	OK	Changed	Removed clinic_address — transitively dependent on clinic, not doctor_id.
PATIENT	Changed	OK	OK	medical_history restructured from comma-separated list to free-text summary.
MEDICINE	OK	OK	OK	All attributes atomic and directly dependent on medicine_id.
APPOINTMENT	OK	OK	OK	Single-column PK; no partial or transitive deps.
PRESCRIPTION	Changed	OK	Changed	Dosage split into 3 atomic cols (1NF). Removed patient_name/doctor_name (3NF).

First Normal Form (1NF)

USER: Satisfies 1NF. All columns hold single atomic values. Role is a single ENUM. user_id uniquely identifies each row. No change needed.

DOCTOR: Satisfies 1NF. availability_start and availability_end are stored as separate TIME columns — not a combined range string. No change needed.

PATIENT: Change required. Early draft stored medical_history as a comma-separated list of conditions. This is non-atomic and violates 1NF. Changed to a free-text TEXT column storing a single summary. A full production system would use a separate PATIENT_CONDITION table.

MEDICINE: Satisfies 1NF. Each column holds exactly one value. medicine_id is the primary key. No change needed.

APPOINTMENT: Satisfies 1NF. scheduled_at is a single DATETIME. status is a single ENUM. appointment_id is the primary key. No change needed.

PRESCRIPTION: Change required. Dosage instructions were stored as one string (e.g. '500mg twice daily for 7 days'). Split into three atomic columns: dosage, frequency, duration_days. This allows independent querying and validation (CHECK duration_days > 0).

Second Normal Form (2NF)

All Tables: All tables satisfy 2NF with no changes. Every table uses a single-column surrogate primary key. Partial dependency — the only 2NF violation — can only occur with composite primary keys, which do not exist in this schema. Every non-key attribute in each table is fully dependent on its table's primary key.

Third Normal Form (3NF)

USER: Satisfies 3NF. No attribute determines another non-key attribute. email, name, phone, role all directly describe user_id. No change needed.

DOCTOR: Change required. Early draft included clinic_address. Since multiple doctors share a clinic, clinic_address depends on the clinic — not doctor_id. This is a transitive dependency. clinic_address was removed. Storing it would create update anomalies: one clinic address change would require updating

every doctor row.

PATIENT: Satisfies 3NF. dob, blood_group, and medical_history all directly describe the patient.

blood_group does not determine any other column. No change needed.

MEDICINE: Satisfies 3NF. type, manufacturer, and description directly describe medicine_id. No transitive chain exists. No change needed.

APPOINTMENT: Satisfies 3NF. scheduled_at, status, and notes directly describe the appointment. Foreign keys patient_id and doctor_id are not source columns for any transitive dependency. No change needed.

PRESCRIPTION: Change required. Early draft included patient_name and doctor_name directly in PRESCRIPTION. These are facts about the patient and doctor — not the prescription — making them transitively dependent via patient_id and doctor_id. Both columns removed; names are retrieved via JOIN with USER through PATIENT and DOCTOR.

Milestone 3 — Dataset Preprocessing & Dataflow

Synthetic Data Generation

Since HAPMS does not use a real-world dataset, structured synthetic data was generated using Python's Faker library. All data is realistic and logically consistent with the schema — names, emails, license numbers, blood groups, appointment slots, and diagnoses all reflect real clinic scenarios.

Table	Rows Generated	Tool Used	Notes
MEDICINE	60	Hardcoded realistic data	Real drug names, types, and manufacturers
USER	110	Python Faker	50 doctors, 55 patients, 5 admins
DOCTOR	50	Python Faker	Real specializations and PMC license numbers
PATIENT	55	Python Faker	Realistic DOB, blood groups, medical histories
APPOINTMENT	120	Python Faker	UNIQUE(doctor_id, scheduled_at) enforced during generation
PRESCRIPTION	74	Python script	Only generated for completed appointments

Dataflow

Where data enters:

Data enters HAPMS through two channels: user-facing registration and booking forms on the frontend, and admin-managed CSV imports during initial data seeding.

How data moves through the system:

- Registration: A new USER row is created with a role. A corresponding DOCTOR or PATIENT row is immediately inserted with role-specific attributes.
- Booking: The patient selects a slot. The system checks APPOINTMENT for an existing (doctor_id, scheduled_at) pair. If none exists, a new APPOINTMENT row is inserted with status = 'pending'.
- Consultation: The doctor updates APPOINTMENT.status: pending → confirmed → completed. At completion, a PRESCRIPTION row is created referencing appointment_id, doctor_id, patient_id, and a medicine_id from the MEDICINE catalogue.
- Querying: The doctor dashboard JOINS APPOINTMENT, PATIENT, and USER. The prescription view JOINS PRESCRIPTION, MEDICINE, DOCTOR, and USER to render a full readable record.

What comes out:

- Patient-facing: Appointment confirmation, prescription with medicine name, dosage, and duration.
- Doctor-facing: Daily schedule, patient history, all issued prescriptions.
- Admin-facing: Weekly report — COUNT(*) of completed appointments grouped by doctor.

Data loading dependency order:

MEDICINE → USER → DOCTOR → PATIENT → APPOINTMENT → PRESCRIPTION

Each table depends on those to its left. This order must be followed to avoid foreign key violations.

Milestone 4 — Database Setup (DDL)

All six tables are created with PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, and CHECK constraints as required. Indexes are added on all foreign key columns and frequently queried columns (status, scheduled_at, issued_date, specialization).

Table	PK	Foreign Keys	UNIQUE	CHECK	Indexes
USER	user_id	—	email	role ENUM	role, email
DOCTOR	doctor_id	user_id	license_no	—	user_id, specialization
PATIENT	patient_id	user_id	—	blood_group ENUM	user_id
MEDICINE	medicine_id	—	name	—	type
APPOINTMENT	appointment_id	patient_id, doctor_id	(doctor_id, scheduled_at)	status ENUM	patient_id, doctor_id, status, scheduled_at
PRESCRIPTION	prescription_id	appointment_id, doctor_id, patient_id, medicine_id	appointment_id	duration_days > 0	patient_id, doctor_id, medicine_id, issued_date

Key Design Decisions

- ON DELETE CASCADE on DOCTOR/PATIENT → USER: deleting a user automatically removes their profile.
- ON DELETE RESTRICT on APPOINTMENT → DOCTOR/PATIENT: prevents deleting a doctor or patient who has booking history.
- UNIQUE(doctor_id, scheduled_at) on APPOINTMENT: database-level guarantee against double-booking.
- UNIQUE(appointment_id) on PRESCRIPTION: enforces one prescription per appointment at the schema level.
- CHECK(duration_days > 0) on PRESCRIPTION: prevents logically invalid prescription durations.

Milestone 5 — Data Population (DML)

Data Loading

Data was loaded using LOAD DATA INFILE statements from the prepared CSV files. Tables were loaded in dependency order: MEDICINE → USER → DOCTOR → PATIENT → APPOINTMENT → PRESCRIPTION.

UPDATE Operations

Update appointment status:

```
UPDATE APPOINTMENT SET status = 'completed' WHERE status = 'confirmed' AND scheduled_at < NOW();
```

Update patient phone:

```
UPDATE USER SET phone = '+92-300-1234567' WHERE user_id = 5 AND role = 'patient';
```

Update doctor hours:

```
UPDATE DOCTOR SET availability_start = '09:00:00', availability_end = '18:00:00' WHERE doctor_id = 3;
```

DELETE Operations

Remove old cancelled appointments:

```
DELETE FROM APPOINTMENT WHERE status = 'cancelled' AND scheduled_at < DATE_SUB(NOW(), INTERVAL 6 MONTH);
```

Remove test pending booking:

```
DELETE FROM APPOINTMENT WHERE appointment_id = 120 AND status = 'pending';
```

Validation Queries & Expected Output

Row count per table:

```
SELECT table_name, COUNT(*) AS row_count from each table via UNION ALL
```

Table	Expected Row Count
USER	110
DOCTOR	50
PATIENT	55
MEDICINE	60
APPOINTMENT	120
PRESCRIPTION	74

Additional validation queries:

Query	What It Checks	Expected Result
NULL check on critical columns	Checks email, role in USER; user_id in DOCTOR/PATIENT; doctor_id/patient_id in APPOINTMENT	All null patient values = 0

Query	What It Checks	Expected Result
Foreign key integrity check	LEFT JOIN PRESCRIPTION → APPOINTMENT — finds orphaned prescriptions	Orphaned prescriptions = 0
Double-booking check	GROUP BY doctor_id, scheduled_at HAVING COUNT(*) > 1 on APPOINTMENT.	APPOINTMENT.
Sample JOIN query	Joins PRESCRIPTION, APPOINTMENT, PATIENT, DOCTOR, USER, MEDICATION.	USER, MEDICATION, PRESCRIPTION
Weekly report query	COUNT(*) of completed appointments per doctor in last 7 days, grouped by doctor_id.	Groups by doctor_id with counts

[GitHub: github.com/YahyaRae](https://github.com/YahyaRae)

Yahya Raees Khan | Eshal Khan | BSAI (B) | DB Systems Lab